Cours PHP — Support résumé

Partie 1 — Créer un site web dynamique

Cette partie présente le principe d'un site dynamique, l'environnement de travail et les bases du langage PHP.

Chapitre P1 — Le principe d'un site web dynamique

P1.1 Page statique et page dynamique

	Page statique	Page dynamique
Fichier	index.html	index.php
Contenu	Identique à chaque visite	Calculé à chaque visite (date, données d'une base, saisie de l'utilisateur)
Traitement	Le serveur envoie le fichier tel quel	Le serveur exécute le script, puis envoie le HTML produit
Exemple	Page « Qui sommes-nous ? »	Catalogue lu dans une base, résultat d'une recherche

P1.2 Le trajet d'une requête

1. Le navigateur (client) demande la page http://localhost/biblio/catalogue.php.
2. Le serveur web (Apache) reçoit la demande et voit l'extension .php.
3. Il confie le fichier à l'interpréteur PHP, qui exécute le script.
4. Si besoin, le script interroge le SGBD (MySQL) et reçoit des données.
5. Le script produit du HTML ; le serveur l'envoie au navigateur.
6. Le navigateur affiche la page. Il ne reçoit jamais le code PHP.

> **À RETENIR**
>
> PHP s'exécute sur le serveur, JavaScript dans le navigateur. Dans le code source de la page (Ctrl + U), on ne voit que le HTML produit, jamais le code PHP.

Chapitre P2 — L'environnement de travail

P2.1 Les outils

Outil	Rôle
XAMPP	Ensemble prêt à l'emploi : serveur web Apache, interpréteur PHP, SGBD MySQL (MariaDB) et phpMyAdmin
Panneau de contrôle XAMPP	Démarre et arrête Apache et MySQL (boutons Start / Stop)

Outil	Rôle
phpMyAdmin	Interface web de gestion des bases : http://localhost/phpmyadmin
Visual Studio Code	Éditeur des fichiers .php
Navigateur	Affiche les pages produites par le serveur

P2.2 Le dossier htdocs et l'adresse localhost

Le serveur ne publie que les fichiers placés dans le dossier `C:\xampp\htdocs`. Créez un dossier par projet, puis ouvrez la page avec l'adresse `http://localhost/dossier/fichier.php`.

```
C:\xampp\htdocs\
└── biblio\
    ├── index.php      →  http://localhost/biblio/index.php
    └── catalogue.php  →  http://localhost/biblio/catalogue.php
```

> **ATTENTION**
>
> Ne jamais ouvrir un fichier .php par un double-clic : le navigateur l'afficherait sans l'exécuter. Passez toujours par http://localhost, avec Apache démarré.

P2.3 Une première page et une première erreur

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Première page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<h1>Bibliothèque municipale</h1>
<?php
echo "<p>Bienvenue sur la page dynamique de la bibliothèque.</p>";
?>
</body>
</html>
```

Quand le script contient une erreur, PHP affiche un message avec le type d'erreur, le fichier et le numéro de la ligne. Ici, un point-virgule manque à la fin de la ligne 2 : PHP signale la ligne 3, là où il rencontre l'instruction inattendue. Sous Windows, le chemin affiché commence par C:\xampp\htdocs.

```
<?php
echo "Bonjour"
echo "à tous";
?>
```

Chapitre P3 — La structure d'un script et l'affichage

P3.1 La structure de base

- `<?php` ouvre un bloc de code PHP, `?>` le ferme. Le reste du fichier est du HTML envoyé tel quel.
- Chaque instruction se termine par un point-virgule.
- **Commentaires** : `// sur une ligne` ou `/* sur plusieurs lignes */`.
- Le fichier porte l'extension .php, même s'il contient surtout du HTML.

P3.2 Afficher avec echo

echo affiche une ou plusieurs expressions, séparées par des virgules. L'opérateur `.` (point) concatène des chaînes. echo peut produire du HTML.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Carte de visite</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
// Carte de visite générée par PHP
$nom = "Leila Ben Salah";
$metier = "Bibliothécaire";
echo "<h1>" . $nom . "</h1>";
echo "<p>", $metier, " · Tunis</p>";
echo '<p>Courriel : leila.bensalah@exemple.tn</p>';
?>
</body>
</html>
```

P3.3 Guillemets doubles ou simples

Écriture	Résultat	Explication
echo "Bonjour $nom";	Bonjour Leila	Entre guillemets doubles, la variable est remplacée par sa valeur
echo 'Bonjour $nom';	Bonjour $nom	Entre guillemets simples, le texte est affiché tel quel
echo 'Bonjour ' . $nom;	Bonjour Leila	Concaténation : fonctionne dans les deux cas

Chapitre P4 — Variables et types

P4.1 Les variables

Une variable commence par `$`, suivi d'un nom (lettre ou _ au début, sans espace ni accent). Elle est créée par sa première affectation avec `=` ; son type dépend de la valeur. La casse compte : $prix et $Prix sont deux variables.

Type	Exemple	Remarque
int (entier)	$quantite = 3;	
float (réel)	$prix = 2.5;	Séparateur décimal : le point
string (chaîne)	$ville = "Tunis";	Entre guillemets
bool (booléen)	$disponible = true;	true ou false
array (tableau)	$notes = [12, 15];	Chapitre P6

P4.2 Opérateurs

Opérateur	Rôle	Exemple	Résultat
+ - * /	Addition, soustraction, multiplication, division	7 / 2	3.5
%	Modulo (reste de la division entière)	7 % 2	1
=	Affectation	$a = 5;	$a vaut 5
.	Concaténation	"Bon" . "jour"	"Bonjour"

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ticket de caisse</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$prixPain = 0.250;
$prixLait = 1.350;
$total = $prixPain * 4 + $prixLait * 2;
echo "<h1>Épicerie du quartier</h1>";
echo "<p>4 pains et 2 litres de lait</p>";
echo "<p>Total à payer : " . $total . " DT</p>";
?>
</body>
</html>
```

P4.3 Le transtypage

Opérateur	Exemple	Résultat
(int)	(int) "12.7 kg"	12
(float)	(float) "12.7 kg"	12.7
(string)	(string) 40	"40"
(bool)	(bool) 0	false
(array)	(array) "Tunis"	["Tunis"]

À RETENIR

Une donnée reçue d'un formulaire ou d'une URL est une chaîne. Le transtypage (int) est très utile pour un identifiant : (int) "3" vaut 3, et une valeur non numérique donne 0.

P4.4 Vérifier qu'une variable existe : isset

isset($variable) retourne true si la variable existe et n'est pas nulle. Il servira surtout à vérifier qu'une donnée a bien été transmise (partie 2).

Chapitre P5 — Les chaînes de caractères

P5.1 Les fonctions de l'annexe

Les positions des caractères commencent à 0.

Fonction	Rôle	Exemple avec $ch = "Carthage"	Résultat
strlen($ch)	Longueur	strlen($ch)	8
substr($ch, d, n)	Sous-chaîne de n caractères à partir de d	substr($ch, 0, 4)	"Cart"
strpos($ch, $m)	Position de la 1re occurrence de $m (false si absent)	strpos($ch, "a")	1
strcmp($a, $b)	Compare : < 0, 0 ou > 0	strcmp("abc", "abd")	< 0
str_replace($m, $r, $ch)	Remplace toutes les occurrences de $m par $r	str_replace("a", "A", $ch)	"CArthAge"
strtolower($ch) · strtoupper($ch)	Minuscules · majuscules	strtoupper($ch)	"CARTHAGE"
trim($ch)	Supprime les espaces au début et à la fin	trim(" Tunis ")	"Tunis"
chr($code) · ord($c)	Caractère d'un code ASCII · code du 1er caractère	chr(65) · ord("a")	"A" · 97

ATTENTION

substr prend une longueur (n caractères), pas une position de fin. str_replace remplace toutes les occurrences, et l'ordre de ses paramètres est : motif, remplacement, chaîne.

P5.2 Exemple : générer un identifiant

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Identifiant</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$prenom = "  Leila ";
$nom = "Ben Salah";
// initiale du prénom + nom sans espace, en minuscules
$login = substr(trim($prenom), 0, 1) . str_replace(" ", "", $nom);
$login = strtolower($login);
echo "<p>Nom : $nom (" . strlen($nom) . " caractères)</p>";
echo "<p>Identifiant : <b>$login</b></p>";
?>
</body>
</html>
```

Chapitre P6 — Les tableaux indicés et associatifs

P6.1 Le tableau indicé

Les éléments sont repérés par un indice numérique qui commence à 0.

```
$notes = array(12, 15.5, 9, 14);   // avec array()
$jours = ["Lundi", "Mardi"];       // écriture courte équivalente
echo $notes[0];                    // 12
$notes[2] = 10;                    // modification
$notes[] = 17;                     // ajout à la fin
echo count($notes);                // 5
```

P6.2 Le tableau associatif

Les éléments sont repérés par une **clé** (une chaîne) au lieu d'un indice. On associe clé et valeur avec `=>`.

```
$livre = [
    "titre" => "Le Petit Prince",
    "auteur" => "Antoine de Saint-Exupéry",
    "annee" => 1943
];
echo $livre["titre"];           // Le Petit Prince
$livre["genre"] = "Roman";      // ajout d'une clé
```

> **À RETENIR**
>
> $_GET, $_POST et chaque ligne lue dans une base de données sont des tableaux associatifs : cette notion est la clé des parties 2 et 3.

P6.3 Parcourir un tableau

Un tableau indicé se parcourt avec for, de 0 à count($t) - 1. Un tableau associatif se parcourt avec `foreach`, qui donne à chaque tour la clé et la valeur.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Relevé de notes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$matieres = ["Mathématiques", "Physique", "Informatique", "Français"];
$notes = [14, 11.5, 17, 12];
$somme = 0;
echo "<table><tr><th>Matière</th><th>Note</th></tr>";
for ($i = 0; $i < count($notes); $i++) {
    echo "<tr><td>" . $matieres[$i] . "</td><td>" . $notes[$i] . "</td></tr>";
    $somme = $somme + $notes[$i];
}
echo "</table>";
echo "<p>Moyenne : " . round($somme / count($notes), 2) . "</p>";
?>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Fiche élève</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$eleve = ["Nom" => "Mansour", "Prénom" => "Yasmine", "Classe" => "4e Info 2", "Ville" =>
"Ariana"];
echo "<table>";
foreach ($eleve as $cle => $valeur) {
    echo "<tr><th>" . $cle . "</th><td>" . $valeur . "</td></tr>";
}
echo "</table>";
?>
</body>
</html>
```

ATTENTION

foreach complète les boucles du programme (for, while, do ... while) : c'est la façon naturelle de parcourir un tableau associatif.

Chapitre P7 — Les structures de contrôle

P7.1 Opérateurs de comparaison et logiques

Comparaison	Signification	Logique	Signification
==	égal à	&&	ET
<> ou !=	différent de	||	OU
< <=	inférieur, inférieur ou égal	!	NON
> >=	supérieur, supérieur ou égal		

> **ATTENTION**
>
> = affecte, == compare. === (identique) compare aussi le type : il est utile avec strpos, qui peut retourner 0 ou false.

P7.2 La structure if

P7.3 Les boucles for, while et do ... while

Boucle	Quand l'utiliser
for ($i = 1; $i <= 10; $i++) { ... }	Nombre de répétitions connu
while (condition) { ... }	Tant qu'une condition est vraie (testée avant : 0 passage possible)
do { ... } while (condition);	Au moins un passage, puis tant que la condition est vraie

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Table de multiplication</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
$n = 7;
echo "<table><tr><th colspan='2'>Table de $n</th></tr>";
for ($i = 1; $i <= 10; $i++) {
    echo "<tr><td>$n × $i</td><td>" . ($n * $i) . "</td></tr>";
}
echo "</table>";
?>
</body>
</html>
```

P7.4 Les fonctions mathématiques

Fonction	Rôle	Exemple	Résultat
abs(x)	Valeur absolue	abs(-7)	7
sqrt(x)	Racine carrée	sqrt(49)	7
round(x, n)	Arrondi (n décimales, facultatif)	round(12.456, 2)	12.46
rand(min, max)	Entier aléatoire entre min et max	rand(1, 6)	1 à 6

Chapitre P8 — La date et l'heure

P8.1 Le fuseau horaire

Pour obtenir l'heure de Tunis, on fixe le fuseau horaire en tête du script avec `date_default_timezone_set("Africa/Tunis");`. Cette instruction ne figure pas dans l'annexe, mais sans elle l'heure affichée peut être décalée.

P8.2 Formater une date avec date()

Caractère	Signification	Exemple
d	Jour du mois sur 2 chiffres	07
m	Mois sur 2 chiffres	10
Y	Année sur 4 chiffres	2026
H	Heure (00 à 23)	14
i	Minutes	05
s	Secondes	42
N	Jour de la semaine (1 = lundi … 7 = dimanche)	3

P8.3 Timestamp, écart entre dates et validité

- `time()` : l'instant présent en timestamp (nombre de secondes écoulées depuis le 1er janvier 1970 à 00:00:00).
- `strtotime("2026-12-20")` : le timestamp d'une date écrite en toutes lettres. La différence de deux timestamps, divisée par 86400 (secondes dans un jour), donne un nombre de jours.
- `date("d/m/Y", $timestamp)` : formate un timestamp donné.
- `checkdate($mois, $jour, $annee)` : true si la date existe. Attention à l'ordre : mois, jour, année.

ceil arrondit à l'entier supérieur : un examen dans 2 jours et 3 heures compte pour J-3. Si vous préférez rester dans l'annexe, round donne un résultat proche.

Chapitre P9 — Importer un fichier avec require

P9.1 Le principe

L'instruction `require "fichier.php";` insère et exécute le contenu d'un autre fichier à cet endroit. Un en-tête modifié une seule fois change sur toutes les pages. Si le fichier est introuvable, require arrête le script avec une erreur.

P9.2 Exemple : un en-tête commun

L'en-tête utilise la variable $titre, définie dans chaque page avant l'import : le titre de l'onglet change d'une page à l'autre.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title><?php echo $titre; ?></title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

> **À RETENIR**
>
> Dans la partie 3, le fichier connexion.php, importé avec require, contiendra la connexion à la base : il sera écrit une seule fois et utilisé par toutes les pages.

Partie 2 — Transmettre des données entre pages

Une page PHP peut recevoir des données d'une autre page : par l'adresse (URL) ou par un formulaire. Ces deux mécanismes préparent la partie 3, où les données reçues serviront à interroger et à modifier une base de données.

Chapitre P10 — Le principe de la transmission

P10.1 Deux méthodes HTTP

	GET	POST
Où sont les données	Dans l'URL, après le ?	Dans le corps de la requête, invisibles dans l'URL
Exemple	livre.php?id=3&format=court	Formulaire avec method="post"
Usage	Lien vers une fiche, recherche, pagination	Saisie d'un formulaire, ajout ou modification de données
Lues dans	$_GET	$_POST

P10.2 Les paramètres d'URL

Après le nom de la page, `?` introduit les paramètres, écrits `nom=valeur` et séparés par `&`.

```
http://localhost/biblio/bonjour.php?prenom=Karim&ville=Sfax
                                   └──────┬──────┘ └──┬──┘
                                  paramètre 1   paramètre 2
```

P10.3 Les variables superglobales

$_GET et $_POST sont des **tableaux associatifs** remplis automatiquement par PHP et disponibles dans toutes les pages. La clé est le nom du paramètre (ou l'attribut name du champ), la valeur est la donnée transmise, toujours sous forme de chaîne.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Bonjour</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<?php
if (isset($_GET["prenom"]) && isset($_GET["ville"])) {
    echo "<p>Bonjour " . $_GET["prenom"] . " de " . $_GET["ville"] . " !</p>";
} else {
    echo "<p>Paramètres manquants.</p>";
}
?>
</body>
</html>
```

ATTENTION

Lire $_GET["prenom"] alors que le paramètre est absent provoque l'avertissement « Undefined array key ». Testez toujours avec isset avant de lire une donnée transmise. Pour arrêter une page à laquelle il manque une donnée indispensable : die("Message");

Chapitre P11 — Transmettre par l'URL avec $_GET

P11.1 Exemple : du catalogue à la fiche

La page catalogue.php génère un lien par livre ; chaque lien transmet l'indice du livre à livre.php. Ici, les livres sont rangés dans un tableau ; dans la partie 3, ils seront lus dans une base de données.

```
<?php
// livres.php : données de démonstration (remplacées par la base en partie 3)
$livres = [
    ["titre" => "Le Petit Prince", "auteur" => "Antoine de Saint-Exupéry", "annee" =>
1943],
    ["titre" => "Les Misérables", "auteur" => "Victor Hugo", "annee" => 1862],
    ["titre" => "Le Prophète", "auteur" => "Khalil Gibran", "annee" => 1923]
];
?>
```

À RETENIR

(int) convertit le paramètre en entier : un id modifié à la main dans l'URL (« abc ») devient 0, puis le test de l'intervalle refuse les valeurs impossibles. Ce réflexe protégera aussi les requêtes SQL de la partie 3.

Chapitre P12 — Transmettre par un formulaire avec $_POST

P12.1 Le formulaire et sa page de traitement

- `action="traitement.php"` : la page qui reçoit les données (vide ou absente : la page elle-même).
- `method="post"` : les données passent par $_POST.
- `name="nom"` : la clé de la donnée dans $_POST. Un champ sans name n'est pas transmis.

P12.2 Exemple : l'inscription à la bibliothèque

Le formulaire et son traitement sont dans la même page. Le bouton porte un name : isset($_POST["envoyer"]) indique que le formulaire vient d'être envoyé.

ATTENTION

Les données d'un formulaire viennent de l'utilisateur : contrôlez-les toujours (champ vide avec trim, nombre avec (int) ou (float), valeur attendue) avant de les afficher ou de les enregistrer dans une base.

P12.3 GET ou POST ?

Situation	Méthode
Lien vers la fiche d'un livre, recherche que l'on veut pouvoir partager	GET
Formulaire d'inscription, de contact, mot de passe	POST
Ajout, modification ou suppression de données dans une base	POST (ou GET avec confirmation, chapitre P18)

Partie 3 — Interagir avec une base de données

C'est la partie principale du cours. Une application web lit et enregistre ses données dans une base MySQL : le catalogue d'une bibliothèque, les produits d'une boutique, les inscrits d'un centre de formation. Chaque chapitre associe une requête SQL à un formulaire : rechercher, ajouter, supprimer, modifier.

Chapitre	Requête	Formulaire ou lien
P13 — La base dans phpMyAdmin	Toutes, en mode assisté	Onglets de phpMyAdmin
P14 — Se connecter	—	—
P15 — Afficher des données	SELECT	Lien vers une fiche ($_GET)
P16 — Rechercher	SELECT … WHERE	Formulaire de recherche ($_GET)
P17 — Ajouter	INSERT	Formulaire d'ajout ($_POST)

Chapitre	Requête	Formulaire ou lien
P18 — Supprimer	DELETE	Lien + formulaire de confirmation
P19 — Modifier	UPDATE	Formulaire pré-rempli ($_POST)
P20 — Application complète	Les quatre	Une page de gestion

Chapitre P13 — La base de données dans phpMyAdmin

P13.1 Rappels

Notion	Définition	Exemple
Base de données	Ensemble de tables gérées par le SGBD	bibliotheque
Table	Ensemble d'enregistrements de même structure	livres
Champ (colonne)	Une information, avec un type	titre VARCHAR(100), annee INT
Enregistrement (ligne)	Un élément de la table	« Le Petit Prince », 1943
Clé primaire	Champ qui identifie chaque ligne de façon unique	id, en AUTO_INCREMENT : numéroté automatiquement

P13.2 La base « bibliotheque »

Dans phpMyAdmin (http://localhost/phpmyadmin), créez la base puis la table, ou collez ce script dans l'onglet SQL. Le script complet du cours, qui crée aussi la base « boutique » des exercices, est fourni par le formateur.

```
CREATE DATABASE bibliotheque CHARACTER SET utf8mb4;
USE bibliotheque;
CREATE TABLE livres (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titre VARCHAR(100) NOT NULL,
  auteur VARCHAR(80) NOT NULL,
  annee INT,
  genre VARCHAR(30),
  disponible TINYINT(1) DEFAULT 1
);
INSERT INTO livres (titre, auteur, annee, genre, disponible) VALUES
('Le Petit Prince', 'Antoine de Saint-Exupéry', 1943, 'Roman', 1),
('Les Misérables', 'Victor Hugo', 1862, 'Roman', 1),
('Le Rouge et le Noir', 'Stendhal', 1830, 'Roman', 0),
('Les Fables', 'Jean de La Fontaine', 1668, 'Poésie', 1),
('Le Bourgeois gentilhomme', 'Molière', 1670, 'Théâtre', 1),
('L''Étranger', 'Albert Camus', 1942, 'Roman', 0),
('Le Prophète', 'Khalil Gibran', 1923, 'Poésie', 1),
('Le Malade imaginaire', 'Molière', 1673, 'Théâtre', 1);
```

La table « adherents » (nom, prenom, email, ville) complète la base ; elle sert à l'exercice de synthèse du chapitre P20.

P13.3 Les requêtes générées en mode assisté

phpMyAdmin construit les requêtes SQL à votre place : vous remplissez un formulaire (onglets Rechercher, Insérer…), il exécute la requête et l'affiche. Il suffit ensuite de la copier dans le script PHP et d'y remplacer les valeurs par des variables.

Requête générée par phpMyAdmin	Version utilisée dans PHP
SELECT * FROM `livres` WHERE `genre` LIKE 'Poésie'	$sql = "SELECT * FROM livres WHERE genre = '$genre'";
INSERT INTO `livres` (`id`, `titre`, …) VALUES (NULL, 'Antigone', …)	$sql = "INSERT INTO livres (titre, …) VALUES ('$titre', …)";

Chapitre P14 — Se connecter et sélectionner la base

P14.1 mysqli_connect

```
$connexion = mysqli_connect("localhost", "root", "", "bibliotheque");
//                          serveur      utilisateur  mot de passe  base
```

- Avec XAMPP : serveur localhost, utilisateur root, mot de passe vide.
- Le 4e paramètre sélectionne la base. On peut aussi la sélectionner ensuite avec mysqli_select_db($connexion, "bibliotheque") (fonction hors annexe).
- mysqli_connect retourne la connexion, ou false en cas d'échec : « or die(…) » arrête alors le script avec un message.
- mysqli_close($connexion) ferme la connexion ; elle est fermée automatiquement à la fin du script.

P14.2 Le fichier connexion.php

```
<?php
// connexion.php : connexion au serveur et sélection de la base
mysqli_report(MYSQLI_REPORT_OFF);   // erreurs gérées par die et mysqli_error
$connexion = mysqli_connect("localhost", "root", "", "bibliotheque")
    or die("Connexion au serveur impossible");
?>
```

> **ATTENTION**
>
> Depuis PHP 8.1, une erreur MySQL arrête le script par défaut avec un message technique, avant que die ou mysqli_error puissent agir. La ligne mysqli_report(MYSQLI_REPORT_OFF); (hors annexe) rétablit le fonctionnement décrit dans ce cours : écrivez-la toujours en tête de connexion.php.

```
<?php
$titre = "Test de connexion";
require "entete.php";
require "connexion.php";
echo "<p class='ok'>Connexion réussie à la base bibliotheque.</p>";
mysqli_close($connexion);
require "pied.php";
?>
```

Si le nom de la base est faux (par exemple « biblioteque »), mysqli_connect échoue : PHP affiche un avertissement qui donne la cause (« Unknown database »), puis die affiche son message et arrête le script.

Chapitre P15 — Afficher des données : SELECT

P15.1 Les étapes

1. Écrire la requête dans une variable : $sql = "SELECT ... FROM ... ";
2. L'exécuter : $resultat = mysqli_query($connexion, $sql);
3. Lire les lignes une à une : while ($ligne = mysqli_fetch_array($resultat)) { ... }
4. Utiliser chaque champ : $ligne["titre"] (par nom) ou $ligne[1] (par indice).

Fonction	Retourne
mysqli_query($connexion, $sql)	Le résultat de la requête, ou false en cas d'erreur
mysqli_fetch_array($resultat)	La ligne suivante, en tableau associatif et indicé ; false quand il n'y en a plus
mysqli_fetch_row($resultat)	La ligne suivante, en tableau indicé seulement
mysqli_num_rows($resultat)	Le nombre de lignes du résultat
mysqli_error($connexion)	Le message d'erreur de la dernière requête

P15.2 Exemple : le catalogue

```
<?php
$titre = "Catalogue";
require "entete.php";
require "connexion.php";
$sql = "SELECT id, titre, auteur, annee, disponible FROM livres ORDER BY titre";
$resultat = mysqli_query($connexion, $sql) or die(mysqli_error($connexion));
echo "<h1>Catalogue (" . mysqli_num_rows($resultat) . " livres)</h1>";
echo "<table><tr><th>Titre</th><th>Auteur</th><th>Année</th><th>Disponible</th></tr>";
while ($ligne = mysqli_fetch_array($resultat)) {
    if ($ligne["disponible"] == 1) { $dispo = "Oui"; } else { $dispo = "Non"; }
    echo "<tr>";
    echo '<td><a href="livre.php?id=' . $ligne["id"] . '">' . $ligne["titre"] .
'</a></td>';
    echo "<td>" . $ligne["auteur"] . "</td><td>" . $ligne["annee"] .
"</td><td>$dispo</td>";
    echo "</tr>";
}
echo "</table>";
mysqli_close($connexion);
require "pied.php";
?>
```

À RETENIR

La boucle while s'arrête toute seule : quand il n'y a plus de ligne, mysqli_fetch_array retourne false. « or die(mysqli_error($connexion)) » affiche l'erreur SQL exacte si la requête est mal écrite.

P15.3 Exemple : la fiche d'un livre

Chaque titre du catalogue est un lien vers livre.php?id=… . La fiche lit l'id dans $_GET, le convertit avec (int), puis sélectionne une seule ligne avec WHERE.

```
<?php
$titre = "Fiche";
require "entete.php";
require "connexion.php";
if (!isset($_GET["id"])) { die("<p class='erreur'>Aucun livre choisi.</p>"); }
$id = (int) $_GET["id"];
$sql = "SELECT * FROM livres WHERE id = $id";
$resultat = mysqli_query($connexion, $sql);
if (mysqli_num_rows($resultat) == 0) { die("<p class='erreur'>Livre introuvable.</p>"); }
$livre = mysqli_fetch_array($resultat);
echo "<h1>" . $livre["titre"] . "</h1>";
echo "<p>" . $livre["auteur"] . ", " . $livre["annee"] . " · " . $livre["genre"] . "</p>";
echo '<p><a href="catalogue.php">Retour au catalogue</a></p>';
require "pied.php";
?>
```

Chapitre P16 — Rechercher avec un formulaire

P16.1 Le principe

Le formulaire de recherche utilise la méthode GET : la recherche apparaît dans l'URL et peut être partagée ou ajoutée aux favoris. Le script lit les critères dans $_GET et les place dans la clause WHERE. `LIKE '%mot%'` trouve les textes qui contiennent « mot » (% remplace n'importe quelle suite de caractères).

P16.2 Exemple : rechercher par genre et par auteur

La liste des genres n'est pas écrite à la main : elle est lue dans la table avec SELECT DISTINCT, qui élimine les doublons. Si l'on ajoute un livre d'un nouveau genre, la liste se met à jour seule.

> **ATTENTION**
>
> Placer directement une saisie dans une requête expose à l'injection SQL : un utilisateur malveillant peut taper du code SQL dans le champ. Dans ce cours : convertissez toujours les identifiants avec (int), limitez les choix par des listes, et ne publiez jamais ces pages sur Internet sans les protections étudiées plus tard (requêtes préparées).

Chapitre P17 — Ajouter des données : INSERT

P17.1 La requête INSERT

```
INSERT INTO livres (titre, auteur, annee, genre)
VALUES ('Antigone', 'Jean Anouilh', 1944, 'Théâtre');
```

- On ne donne pas l'id : AUTO_INCREMENT le calcule.
- Textes entre apostrophes, nombres sans apostrophes, dans le même ordre que les champs.
- mysqli_query retourne true si la requête a réussi ; mysqli_affected_rows($connexion) donne le nombre de lignes ajoutées.

P17.2 Exemple : le formulaire d'ajout

P17.3 Le piège de l'apostrophe

En SQL, l'apostrophe délimite les textes. Le titre « L'Alchimiste » ferme donc le texte trop tôt et la requête devient incorrecte. On double l'apostrophe avec `str_replace("'", "''", $texte)` : c'est l'écriture SQL d'une apostrophe dans un texte. Sans cette ligne, mysqli_error affiche l'erreur :

```
$sql = "INSERT INTO livres (titre, auteur, annee, genre)
        VALUES ('$t', '$a', $annee, '$genre')";
// avec $t = "L'Alchimiste" : VALUES ('L'Alchimiste', …) est incorrect
```

Chapitre P18 — Supprimer des données : DELETE

P18.1 La requête DELETE

```
DELETE FROM livres WHERE id = 3;
```

> **ATTENTION**
>
> Sans clause WHERE, DELETE FROM livres vide toute la table, sans retour possible. Une suppression se fait toujours sur la clé primaire : WHERE id = $id, avec $id converti par (int).

P18.2 La liste de gestion

La page gestion.php affiche les livres avec, pour chacun, un lien « Modifier » et un lien « Supprimer » qui transmettent l'id par l'URL.

```
<?php
$titre = "Gestion du catalogue";
require "entete.php";
require "connexion.php";
$resultat = mysqli_query($connexion, "SELECT id, titre, auteur, annee, disponible FROM
livres ORDER BY id")
    or die(mysqli_error($connexion));
echo "<h1>Gestion du catalogue</h1>";
echo '<p><a href="ajout.php">+ Ajouter un livre</a></p>';
echo "<table><tr><th>N°</th><th>Titre</th><th>Auteur</th><th>Dispo.</th><th>Actions</th></
tr>";
while ($l = mysqli_fetch_array($resultat)) {
    if ($l["disponible"] == 1) { $d = "Oui"; } else { $d = "Non"; }
    echo "<tr><td>" . $l["id"] . "</td><td>" . $l["titre"] . "</td><td>" . $l["auteur"] .
"</td><td>$d</td>";
    echo '<td><a href="modifier.php?id=' . $l["id"] . '">Modifier</a> | ';
    echo '<a href="supprimer.php?id=' . $l["id"] . '">Supprimer</a></td></tr>';
}
echo "</table><p>" . mysqli_num_rows($resultat) . " livres</p>";
require "pied.php";
?>
```

P18.3 La page de suppression avec confirmation

supprimer.php lit l'id dans $_GET et affiche d'abord une question. La suppression n'a lieu qu'après le clic sur « Oui, supprimer », qui envoie le formulaire en POST à la même adresse (l'id reste dans l'URL).

Chapitre P19 — Modifier des données : UPDATE

P19.1 La requête UPDATE

```
UPDATE livres SET annee = 1943, disponible = 1 WHERE id = 1;
```

ATTENTION

Comme DELETE, UPDATE sans WHERE modifie toutes les lignes de la table. Modifiez toujours une ligne précise : WHERE id = $id.

P19.2 Le formulaire pré-rempli

La page modifier.php reçoit l'id par l'URL, lit la ligne avec SELECT et place chaque valeur dans l'attribut value du champ : `value="<?php echo $livre["titre"]; ?>"`. Une case est pré-cochée en écrivant checked ; une option, présélectionnée avec selected.

À RETENIR

mysqli_affected_rows vaut 0 si l'on enregistre sans rien changer : MySQL ne compte que les lignes réellement modifiées. Ce n'est pas une erreur.

Chapitre P20 — Une application complète

P20.1 L'organisation des fichiers

```
htdocs\biblio\
├── connexion.php   connexion à la base (importé partout)
├── entete.php      début de page commun, titre variable
├── pied.php        fin de page commune
├── style.css
├── gestion.php     liste + liens Ajouter, Modifier, Supprimer   (SELECT)
├── ajout.php       formulaire d'ajout                           (INSERT)
├── modifier.php    formulaire pré-rempli                        (SELECT + UPDATE)
└── supprimer.php   confirmation                                 (SELECT + DELETE)
```

P20.2 Récapitulatif

Action	Déclencheur	Données reçues	Requête	Vérification
Lister	Ouverture de gestion.php	—	SELECT	mysqli_num_rows

Action	Déclencheur	Données reçues	Requête	Vérification
Rechercher	Formulaire GET	$_GET	SELECT … WHERE	mysqli_num_rows
Ajouter	Formulaire POST	$_POST	INSERT	mysqli_affected_rows
Modifier	Lien ?id= puis formulaire POST	$_GET["id"] + $_POST	UPDATE … WHERE id	mysqli_affected_rows
Supprimer	Lien ?id= puis confirmation POST	$_GET["id"]	DELETE … WHERE id	mysqli_affected_rows

P20.3 Le parcours complet

Les quatre pages des chapitres P17 à P19 forment l'application. Voici la liste après un ajout, une modification et une suppression :

Annexe — Mémo PHP

Ce mémo reprend l'annexe officielle du programme et les compléments utilisés dans le cours (signalés « hors annexe »).

Types, opérateurs et affichage

Élément	Syntaxe
Types	int, float, string, bool, array
Transtypage	(int) (float) (string) (bool) (array)
Arithmétiques	+ - * / % (modulo)
Comparaison	== <> (ou !=) < <= > >= ; === identique (hors annexe)
Logiques	&& (et) || (ou) ! (non)
Affectation, concaténation	= .
Affichage	echo expression1, expression2;

Fonctions

Domaine	Fonctions
Chaînes	chr, ord, strlen, substr, strpos, strcmp, str_replace, strtolower, strtoupper, trim
Mathématiques	abs, sqrt, round, rand
Tableaux	array, count ; parcours foreach (hors annexe)
Date et heure	checkdate, date, time, strtotime ; date_default_timezone_set (hors annexe)
Diverses	die, isset, require
Superglobales	$_GET (paramètres d'URL), $_POST (formulaire en méthode POST)

MySQLi, interface procédurale

Fonction	Rôle
mysqli_connect(serveur, utilisateur, mdp, base)	Connexion au serveur et sélection de la base
mysqli_query($connexion, $sql)	Exécute une requête : résultat (SELECT) ou true / false
mysqli_fetch_array($resultat)	Ligne suivante, tableau associatif et indicé
mysqli_fetch_row($resultat)	Ligne suivante, tableau indicé
mysqli_num_rows($resultat)	Nombre de lignes d'un résultat
mysqli_affected_rows($connexion)	Lignes ajoutées, modifiées ou supprimées par la dernière requête
mysqli_error($connexion)	Message d'erreur de la dernière requête
mysqli_close($connexion)	Ferme la connexion
mysqli_report(MYSQLI_REPORT_OFF)	Hors annexe : à placer avant la connexion (PHP 8.1 et suivants)
mysqli_select_db($connexion, base)	Hors annexe : sélection de la base après la connexion

Requêtes SQL

```
SELECT champs FROM table WHERE condition ORDER BY champ;
INSERT INTO table (champ1, champ2) VALUES ('texte', 12);
UPDATE table SET champ1 = 'texte', champ2 = 12 WHERE id = 3;
DELETE FROM table WHERE id = 3;
```